MODULE 13 · DOMAIN 4 - TERRAFORM CONFIGURATION

# Sensitive Data, Ephemeral Values, Write-Only Arguments

The problem raised in module four, and the mechanisms that finally solve it.

DOMAIN 4 of 8 COURSE WEIGHT 24% MODULE 13 of 27

admin@sthithapragna.com

**sensitive** hides a value. **Ephemeral** means the value is never written down at all.

# Where secrets ended up, before all this

- **State.** Every attribute a provider returns, in plain text. Module four.
- **Saved plan files.** Full configuration and all planned values, in cleartext. Module five.
- **Outputs**, which are stored in state, and revealed by output -raw or -json. Module eight.
- HashiCorp's response was blunt: **treat your state as sensitive data**, and they still advise it.
- **sensitive = true never changed any of this.** It suppresses display, and nothing more.

> ON THE EXAM
>
> Three modules made this point separately. It is one problem with one shape, and it has one root: the value was persisted.

# The mitigations that came first

- **Encrypt state at rest**, and use a backend that supports it. Necessary, and not sufficient.
- **Restrict who can read the backend.** Anyone with read access sees plain text, encryption or not.
- **Never commit state, tfvars files or plan files.** All three are ordinary text.
- **Supply secrets at run time**, through environment variables or a secrets manager, rather than in files.
- **Rotate anything that has been exposed.** Removing a value from a file does not un-expose it.

> ON THE EXAM
>
> These remain correct and remain necessary. They reduce who can see the secret. They never stopped it being written down.

# Vault, and why a data source is not the answer

- The objective names **secrets management with Vault**, and the Vault provider is the obvious tool.
- **But reading a secret with a data source puts it in state.** A data source result is stored like any other.
- So the classic pattern moved the secret out of your repository and **straight into your state file**.
- It was still an improvement, because the secret was centrally managed and rotatable.
- **An ephemeral resource is the fix**, and Vault ships them. Same source, no persistence.

> ON THE EXAM
>
> This is the trap in the traditional answer. Fetching from a secrets manager solves distribution. It never solved persistence.

# Ephemerality

- An **ephemeral** value exists only for the duration of the current operation.
- **It is never written to the plan artefact or the state file.** That is the definition, not a side effect.
- Introduced in **Terraform 1.10** for input variables, output values, and a new kind of resource.
- Extended in **Terraform 1.11** so ephemeral values could reach ordinary managed resources.
- The exam tests **1.12**, so both halves are in scope.

> ON THE EXAM
>
> Two releases, two halves. 1.10 created ephemeral values. 1.11 gave them somewhere useful to go.

# Ephemeral variables and outputs

- A variable or output marked **ephemeral = true** is not persisted anywhere.
- **An ephemeral output is not stored in state**, so terraform output cannot read it afterwards.
- That is the point. It can be consumed within the run, and it leaves no record.
- **In 1.10, ephemeral values could only flow into other ephemeral contexts.**
- So a token could authenticate a provider, and could not be passed to a normal resource argument.

> ON THE EXAM
>
> sensitive and ephemeral are different arguments answering different questions. A value can be both.

# Ephemeral resources

- A third resource mode, declared with an **ephemeral** block, alongside managed resources and data sources.
- Terraform **opens** it, uses it, optionally **renews** it, then explicitly **closes** it before the phase ends.
- **They are executed during every plan and every apply.** A new value each time.
- They are **nodes in the dependency graph**, so anything depending on one is provisioned after it.
- Examples: a generated random password, a fetched secret version, a short-lived token.

> ON THE EXAM
>
> Three resource modes now: managed resources create things, data sources read things, ephemeral resources borrow things.

# Open, renew, close

- Terraform **opens** an ephemeral resource when its value is first needed in a phase.
- It **renews** it if the provider says the value is leased and would otherwise expire mid-run.
- It **closes** it explicitly before the phase ends, so a temporary credential is revoked rather than abandoned.
- That is why these suit **dynamically issued credentials**, not just static secrets.
- A plan and an apply are **separate phases**, so the cycle happens twice across a normal workflow.

> ON THE EXAM
>
> The close step is the part that distinguishes this from just 'not storing it'. The credential is actively given back.

# Data source or ephemeral resource?

DATA SOURCE

## Reads, and remembers

- Read during plan, or deferred to apply
- Result stored in state
- Other resources reference its attributes
- Right for configuration you look up

EPHEMERAL RESOURCE

## Borrows, and forgets

- Opened, used, closed within the operation
- Nothing stored, ever
- Feeds providers and write-only arguments
- Right for secrets and credentials

THE TIEBREAKER

**Both read from somewhere else. The difference is entirely about what happens afterwards.**

# Write-only arguments

- **Terraform 1.11.** Managed resource attributes that are **not persisted to the plan or state artefacts**.
- Named by convention with a **_wo** suffix, such as password_wo on a database instance.
- **The provider is the terminal point.** It uses the value to call the API, then Terraform discards it.
- **Unlike other ephemeral constructs, they accept ephemeral and non-ephemeral values.**
- **Not usable with set attributes, set nested attributes or set nested blocks.**

> ON THE EXAM
>
> This is the bridge. Ephemeral values could not reach a managed resource until a write-only argument gave them a door.

# The version argument, and why it exists

- Terraform does not store a write-only value, so **it cannot detect when that value changed**.
- So write-only arguments are **sent to the provider on every operation**, regardless.
- Providers therefore pair them with a **version argument**, such as password_wo_version.
- **The version IS stored in state.** Incrementing it is how you tell Terraform to accept a new value.
- With several write-only arguments on one resource, **keep their versions in sync** so they update together.

> ON THE EXAM
>
> Ephemeral resources run every time, so without the version a password would appear to change on every apply. The version is the brake.

# End-to-end ephemerality

01

## Generate

An ephemeral resource produces a random password. Nothing is stored.

02

## Store

Write it to a secrets manager using a write-only argument.

03

## Fetch

Read it back with an ephemeral resource during apply.

04

## Use

Pass it to the database's write-only password argument.

ON THE EXAM

The secret is generated, stored, retrieved and used without ever appearing in the plan file or state. That is the documented pattern.

# Finding out what supports it

- **Providers indicate in the Terraform Registry whether an argument is write-only.**
- **Support is per resource, per provider.** Not every sensitive argument has a write-only counterpart.
- Ephemeral resources are likewise **published by providers**, and listed in the registry.
- So adopting this is partly a **provider version** question, which is module three all over again.
- You also need **Terraform 1.11 or later** for write-only, and 1.10 for ephemeral resources.

> ON THE EXAM
>
> Two version constraints, on two different pieces of software. The Terraform version and the provider version must both be new enough.

# Does this end up in state?

| THE VALUE | IN STATE? |
|---|---|
| A resource attribute returned by the provider | Yes, in plain text |
| A variable marked sensitive | Yes. sensitive is display only |
| A data source result, from Vault or anywhere | Yes |
| An output marked sensitive | Yes, and redacted when printed |
| An ephemeral resource's value | No, never |
| A write-only argument's value | No. But its version argument is stored |

ON THE EXAM

Five of these six were true before 1.10, and the last two are the whole of what changed.

# What this does not do

- **It does not clean your existing state.** A password already recorded there stays exactly where it is.
- **It does not retrospectively protect anything.** Adoption changes future runs only.
- So adopting it should be paired with **rotating the secrets that were already exposed**.
- **It does not replace access control or encryption at rest.** State still holds plenty worth protecting.
- HashiCorp still advises **treating state as sensitive data**, after all of this.

> ON THE EXAM
>
> A secret in state is exposed the moment it is written. Nothing added later un-writes it, which is the same logic as a Git commit.

# Which mechanism, for which requirement

| THE REQUIREMENT | THE MECHANISM |
| --- | --- |
| Do not print this in the terminal | sensitive = true |
| Do not store this in state at all | ephemeral, or a write-only argument |
| Fetch a secret without persisting it | An ephemeral resource |
| Pass a secret to a normal resource | A write-only argument on that resource |
| Change the stored secret deliberately | Increment the write-only version argument |
| Protect what is already in state | Rotate it. Nothing else works |

ON THE EXAM

sensitive and ephemeral are not alternatives. One controls display, the other controls persistence, and a value can be both.

A team stores database credentials in Vault and reads them into Terraform with the Vault provider's data sources, believing this keeps the passwords out of their configuration.

A security review finds the passwords in plain text in the remote state file. The team is on Terraform 1.12 with a current Vault provider.

## What should they change?

A Mark the data source's output as sensitive, which prevents it being written to state

B Replace the data source with an ephemeral resource, which is never persisted

C Enable encryption at rest on the backend, which resolves the finding

D Move the credentials to TF_VAR_ environment variables instead of Vault

◆ PAUSE AND DECIDE

## What should they change?

**A** Mark it sensitive

*The misconception this course has flagged in four modules. sensitive suppresses display; it has never affected storage.*

**B** Use an ephemeral resource

Correct. Same secret, same source, and the value is never written to the plan artefact or state. Available on 1.12 with a current provider.

**C** Encrypt at rest

*Worth doing anyway, and it does not resolve this. The reviewer had legitimate read access, and encryption does nothing about that.*

**D** Environment variables

*Moves the secret out of Vault into a pipeline configuration, losing central management and rotation. The value would still land in state.*

THE SIGNAL

Fetching from a secrets manager solved distribution and never solved persistence. Ephemeral resources are what closed that gap.

> An engineer configures a database with an ephemeral random_password resource feeding the password_wo argument. She sets password_wo_version to 1 and applies successfully.
>
> Security asks for the password to be rotated. She reruns apply, expecting the ephemeral resource to generate a new password. The plan reports no changes.

## What is happening?

A The ephemeral resource caches its value in state after the first apply

B The write-only version argument is unchanged, so Terraform does not accept a new value

C Write-only arguments can only be set at creation and never updated

D The plan needs -replace on the database to pick up the new password

◆ PAUSE AND DECIDE

## What is happening?

A **It caches in state**

*Ephemeral resources persist nothing, by definition, and they execute on every plan and apply. A new password genuinely was generated.*

B **The version is unchanged**

Correct. Terraform cannot see the value, so the version argument stored in state is the only signal it has. Increment it to rotate.

C **Create-only**

*False. Updating is exactly what the version mechanism exists to support.*

D **Use -replace**

*Would destroy and recreate the database to change one argument. The version increment updates it in place.*

THE SIGNAL

The ephemeral resource did generate a new password. Terraform simply had no reason to send it, because nothing it can see had changed.

A platform team migrates twelve configurations to use ephemeral resources and write-only arguments. The work is completed and reviewed, and every new run keeps secrets out of state.

They close the security finding, noting that credentials are no longer present in Terraform state.

## What is wrong with closing the finding?

A. Nothing. Once write-only arguments are in use, previous values are purged on the next apply

B. Existing state still contains the old credentials. They must be rotated, and old state versions considered

C. Write-only arguments only take effect after terraform init -upgrade is run

D. The finding should stay open until every provider supports write-only arguments

◆ PAUSE AND DECIDE

## What is wrong with closing the finding?

A. Old values are purged

*Nothing purges them. Adoption changes what future runs write; it never rewrites what past runs already wrote.*

B. Rotate, and check history

Correct. The credentials were exposed the moment they were written, and a versioned backend keeps earlier state versions containing them.

C. Needs init -upgrade

*Confuses provider installation with data handling. -upgrade re-resolves versions and touches no stored values.*

D. Wait for all providers

*Would keep the finding open indefinitely for no benefit. Partial adoption genuinely reduces exposure going forward.*

THE SIGNAL

Exactly module one's Git logic, one layer down. Removing a secret from future writes does not un-write the past ones.

A configuration authenticates a third-party provider using a long-lived API token, currently supplied through an environment variable and shared across the team.

The team wants a short-lived token issued per run, from their secrets platform, with nothing stored in Terraform's artefacts. They are on Terraform 1.12.

## What approach fits?

A. A data source that fetches the token, referenced from the provider block

B. An ephemeral resource fetching the token, referenced from the provider block

C. A write-only argument on the provider block

D. A sensitive variable holding the token, supplied by the pipeline

◆ PAUSE AND DECIDE

## What approach fits?

A. A data source

*Fetches the token and stores it in state, which is scenario 1's problem again. It also cannot issue a per-run credential.*

B. An ephemeral resource

Correct. Provider configuration is an ephemeral context, so an ephemeral resource can populate it, and the token is opened, used and closed within the run.

C. Write-only on the provider

*Write-only arguments are a feature of managed resources. A provider block does not need them, because it is already ephemeral.*

D. A sensitive variable

*Keeps the long-lived shared token and only hides it in output. It addresses neither the lifetime nor the storage requirement.*

THE SIGNAL

Provider configuration was ephemeral from the start, in 1.10, before write-only arguments existed. Authentication was the first use case.

> A module author is reviewing three requirements. First, an admin password must not appear in state. Second, a cost centre code should not be printed in plan output, though storing it is fine. Third, the module must work for consumers on Terraform 1.9.
>
> She asks which mechanisms to use for the first two, and what the third constraint implies.

## What is the correct assessment?

A. Use ephemeral for both, and 1.9 is sufficient for both features

B. Write-only for the password, sensitive for the cost centre, and 1.9 consumers cannot use the write-only argument

C. Use sensitive for both, since sensitive covers storage and display

D. Use write-only for both, since it supersedes sensitive entirely

◆ PAUSE AND DECIDE

## What is the correct assessment?

**A** Ephemeral for both

*Overreaches on the second requirement and gets the version wrong. Ephemeral resources need 1.10; the module targets 1.9.*

**B** Write-only, sensitive, and a version problem

Correct. Different requirements, different mechanisms, and write-only arguments need 1.11 or later, which 1.9 consumers do not have.

**C** sensitive for both

*The running misconception. sensitive controls display only, so the password would still be written to state in plain text.*

**D** Write-only for both

*They are not alternatives. The cost centre must be stored, and a write-only argument would discard it, breaking the resource.*

THE SIGNAL

Not every secret needs the strongest tool. A value that must be stored but not displayed is exactly what sensitive is for.

# Module 13 recap

- **sensitive hides. Ephemeral never writes it down.** Different questions, and a value can be both.
- **1.10 gave ephemeral variables, outputs and resources.** They could only feed other ephemeral contexts.
- **1.11 added write-only arguments**, the door into ordinary managed resources.
- **Ephemeral resources run on every plan and apply**, which is why the version argument exists.
- **The version is stored in state.** Increment it to rotate a write-only value.
- **None of it cleans existing state.** Rotate what was already exposed.

> ON THE EXAM
>
> A Vault data source still lands in state. An ephemeral resource does not. That single swap closes most of the gap.